

PROJETO FINAL: VALIDADOR FORMAL EM TRÊS NÍVEIS

Tema escolhido: Tema 1: Validador de cadastro, fórmulas e integridade

Relatório Técnico

Mariana Marques Braguim
Caio Willian de Melo Runpfe

1. Introdução

Neste trabalho, desenvolvemos um validador formal baseado em três níveis da Hierarquia de Chomsky. A ideia foi implementar, dentro do mesmo tema, três reconhecedores com poderes computacionais diferentes: um DFA para validar o formato textual de CPF, um PDA para verificar delimitadores balanceados em expressões simbólicas e uma Máquina de Turing para reconhecer strings no formato $L = \{w\#w \mid w \in \{0,1\}^*\}$.

O objetivo principal é comparar, na prática, como cada modelo reconhece uma linguagem diferente. O DFA resolve padrões regulares sem memória auxiliar, o PDA usa uma pilha para controlar aninhamentos e a Máquina de Turing utiliza uma fita com leitura e escrita para comparar partes distintas da entrada.

Com isso, o trabalho mostra de forma aplicada a relação entre os três níveis:

$$LR \subsetneq LLC \subsetneq R$$

2. Linguagens escolhidas

2.1 Linguagem Regular: Validação de CPF

Para a parte regular, usamos a validação do formato textual de CPF no padrão ddd.ddd.ddd-dd. O reconhecedor não calcula os dígitos verificadores do CPF; ele apenas confere se a entrada segue a estrutura esperada, com onze dígitos, dois pontos e um hífen nas posições corretas.

Essa linguagem é regular porque a decisão depende apenas da posição de cada símbolo lido. O reconhecedor não precisa guardar informações arbitrárias nem comparar partes diferentes da string.

Formalmente:

$$LCPF = \{d_1d_2d_3.d_4d_5d_6.d_7d_8d_9 - d_{10}d_{11} \mid d_i \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}\}$$

Alfabeto:

$$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ., -, \}$$

Aceita	Rejeita
123.456.789-00	12345678900
000.000.000-00	12.345.678-90
987.654.321-99	123.456.789.00

O modelo escolhido foi um Autômato Finito Determinístico, pois cada estado representa uma posição esperada da entrada. Se qualquer símbolo aparecer fora da posição correta, o autômato vai para um estado de erro.

2.2 Linguagem Livre de Contexto: Delimitadores balanceados

Na parte livre de contexto, o objetivo é verificar se os delimitadores de uma expressão estão balanceados. São considerados os pares $()$, $[]$ e $\{\}$, permitindo também letras, números e operadores no interior da expressão.

Esse problema exige memória auxiliar, porque o reconhecedor precisa lembrar cada símbolo de abertura encontrado para conferir se o fechamento correspondente aparece depois, na ordem correta. Por isso, usamos um Autômato com Pilha.

Formalmente:

$$LDEL = \{w \in \Sigma^* \mid \text{os delimitadores } (), [], \{\} \text{ em } w \text{ estão corretamente balanceados}\}$$

Alfabeto:

$$\Sigma = \{(), [], \{\}, \text{letras, dígitos, } +, -, *, /\}$$

Aceita	Rejeita
(a+b)	((a+b)
((x+y)*z)	[a+b))
{{(x+y)}}	{{(a+b)}}

O modelo escolhido foi um PDA, pois a pilha permite armazenar os símbolos de abertura e verificar se cada fechamento corresponde ao topo da pilha.

2.3 Linguagem Recursiva: Cópia de string

Na linguagem recursiva, trabalhamos com strings no formato $w#w$. A Máquina de Turing precisa verificar se o trecho antes do símbolo # é exatamente igual ao trecho depois dele. Para isso, ela marca os símbolos já comparados com X e percorre a fita várias vezes até aceitar ou rejeitar a entrada.

Formalmente:

$$LWW = \{w#w \mid w \in \{0,1\}^*\}$$

Alfabeto de entrada:

$$\Sigma = \{0, 1, \#\}$$

Alfabeto da fita:

$$\Gamma = \{0, 1, \#, X, \square\}$$

O símbolo X representa uma posição já verificada, e $\square$ representa o branco da fita.

Aceita	Rejeita
#	0#1
0#0	101#100
101#101	01#011

A string # é aceita porque representa o caso $w = \varepsilon$, ou seja, a string vazia antes e depois do separador.

3. Modelos formais e tabelas de transição

3.1 DFA para validação de CPF

O DFA reconhece entradas no formato ddd.ddd.ddd-dd. O conjunto de estados é $Q = \{q_0, q_1, q_2, \dots, q_{14}, q_{err}\}$. O estado inicial é q_0 e o único estado final é q_{14} .

D representa qualquer dígito de 0 a 9.

Estado	Entrada	Próximo estado
q0	D	q1
q1	D	q2
q2	D	q3
q3	.	q4
q4	D	q5
q5	D	q6
q6	D	q7
q7	.	q8
q8	D	q9
q9	D	q10
q10	D	q11
q11	-	q12
q12	D	q13
q13	D	q14
q0 a q14	símbolo não previsto	q_err
q_err	qualquer símbolo	q_err

Estado inicial: q_0

Estado final: $F = \{q_{14}\}$

Diagrama do DFA:

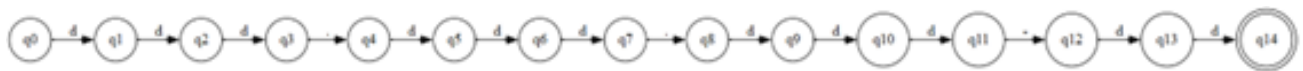


Figura 1: Autômato finito determinístico para validação do formato textual de CPF.

As transições entre os estados seguem a ordem exata do formato do CPF. Em q3 e q7 é esperado ponto, em q11 é esperado hífen, e nas demais posições são esperados dígitos. Qualquer símbolo diferente do esperado leva o DFA para q_err.

3.2 PDA para delimitadores balanceados

O PDA possui um estado principal de processamento q0, um estado final qf e um estado de erro q_err. A pilha começa com o símbolo de fundo Z.

Quando o PDA lê um símbolo de abertura, ele empilha esse símbolo. Quando lê um fechamento, ele só desempilha se o topo da pilha for o símbolo de abertura correspondente. Letras, dígitos e operadores são apenas consumidos, sem alterar a pilha.

Estado	Entrada	Topo da pilha	Ação	Próximo estado
q0	(	qualquer	empilha (	q0
q0	[	qualquer	empilha [	q0
q0	{	qualquer	empilha {	q0
q0	)	(	desempilha	q0
q0	]	[	desempilha	q0
q0	}	{	desempilha	q0
q0	símbolo comum	qualquer	mantém pilha	q0
q0	ϵ	Z	mantém Z	qf
q0	fechamento incompatível	qualquer	rejeita	q_err

Estado inicial: q0

Estado final: qf

Símbolo de fundo da pilha: Z

Diagrama do PDA:

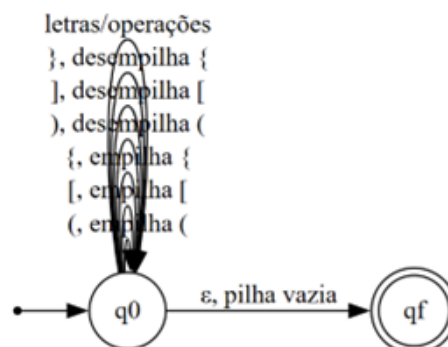


Figura 2: Autômato com pilha para validação de delimitadores balanceados.

q0 consome a entrada, empilhando aberturas e desempilhando fechamentos corretos. Quando a leitura termina e a pilha contém apenas Z, ocorre uma transição ϵ para qf. Se um fechamento não corresponder ao topo da pilha, a entrada é rejeitada.

3.3 Máquina de Turing para $L = \{w\#w \mid w \in \{0,1\}^*\}$

A Máquina de Turing compara a parte esquerda e a parte direita da entrada. Para isso, marca com X os símbolos já verificados. A cada ciclo, ela escolhe o próximo símbolo não marcado antes do #, procura o símbolo correspondente depois do #, marca esse símbolo e retorna ao início da fita.

Estado	Lê	Escreve	Move	Próximo
q0	X	X	R	q0
q0	0	X	R	q1
q0	1	X	R	q2
q0	#	#	R	q6
q0	□	□	R	qr
q1	0	0	R	q1
q1	1	1	R	q1
q1	X	X	R	q1
q1	#	#	R	q3
q1	□	□	R	qr
q2	0	0	R	q2
q2	1	1	R	q2
q2	X	X	R	q2
q2	#	#	R	q4
q2	□	□	R	qr
q3	X	X	R	q3
q3	0	X	L	q5
q3	1	1	R	qr
q3	#	#	R	qr
q3	□	□	R	qr
q4	X	X	R	q4
q4	1	X	L	q5
q4	0	0	R	qr
q4	#	#	R	qr
q4	□	□	R	qr
q5	0	0	L	q5
q5	1	1	L	q5
q5	X	X	L	q5
q5	#	#	L	q5
q5	□	□	R	q0
q6	X	X	R	q6
q6	0	0	R	qr
q6	1	1	R	qr
q6	#	#	R	qr
q6	□	□	R	qa

Descrição dos estados:

q0: procura o próximo símbolo não marcado da primeira parte.

q1: registra que o símbolo escolhido foi 0 e avança até o lado direito.

q2: registra que o símbolo escolhido foi 1 e avança até o lado direito.

q3: procura um 0 correspondente no lado direito.

q4: procura um 1 correspondente no lado direito.

q5: retorna ao início da fita.

q6: verifica se não sobraram símbolos não marcados no lado direito.

qa: estado de aceitação.

qr: estado de rejeição.

Diagrama da Máquina de Turing:

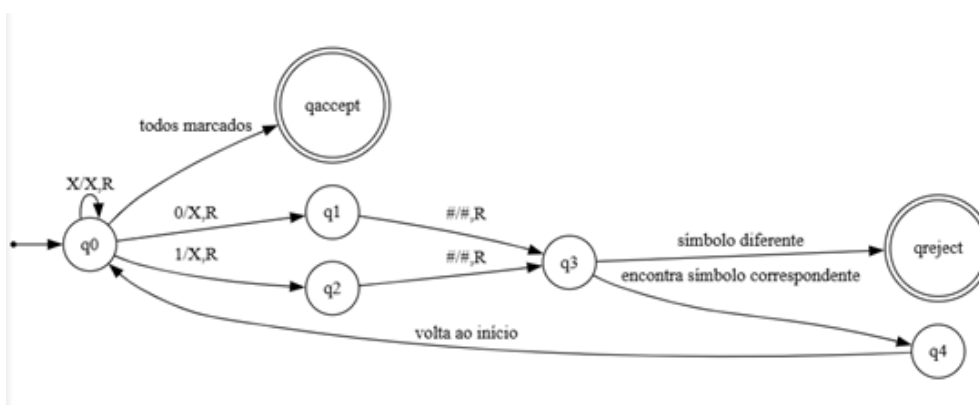


Figura 3: Máquina de Turing para reconhecimento da linguagem $L = \{w\#w \mid w \in \{0,1\}^*\}$.

q0 seleciona símbolos da primeira parte. Quando escolhe 0, vai para q1 e depois q3 para procurar um 0 correspondente. Quando escolhe 1, vai para q2 e depois q4 para procurar um 1 correspondente. Após marcar o símbolo correspondente, q5 retorna ao início da fita. Quando q0 encontra #, a máquina entra em q6 para verificar se todos os símbolos do lado direito já foram marcados. Se estiverem todos marcados, aceita; caso contrário, rejeita.

4. Implementação

Os três reconhecedores foram implementados manualmente em Python. Em todos os casos, os estados, alfabetos, estados finais e tabelas de transição foram declarados explicitamente como estruturas de dados. O módulo `re` não foi usado como mecanismo principal de reconhecimento.

No DFA do CPF, a entrada é processada símbolo a símbolo. A cada leitura, o estado atual é atualizado de acordo com a função de transição. Símbolos inesperados levam ao estado `q_err`, e a entrada só é aceita se a leitura terminar em `q14`.

No PDA dos delimitadores, usamos uma lista Python para representar a pilha. Símbolos de abertura são empilhados, símbolos de fechamento desempilham apenas quando correspondem ao topo da pilha, e letras, dígitos ou operadores são apenas consumidos. A entrada é aceita quando a leitura termina e resta apenas o símbolo de fundo `Z`.

Na Máquina de Turing para $w#w$, a fita é representada por uma lista. A máquina marca com `X` os símbolos já comparados, procura o correspondente no lado direito da string e retorna ao início até que todos os símbolos sejam processados. A entrada é aceita apenas quando as duas partes possuem exatamente os mesmos símbolos na mesma ordem.

A definição de passo adotada segue o enunciado: no DFA, cada leitura de símbolo com mudança de estado conta como um passo; no PDA, cada transição, incluindo empilhamento, desempilhamento ou transição ϵ , conta como um passo; na Máquina de Turing, cada leitura, escrita e deslocamento da cabeça conta como um passo.

5. Testes e resultados

A bateria de testes contém 18 entradas no total: 3 aceitas e 3 rejeitadas para cada uma das três linguagens. Para cada teste, registramos o resultado esperado, o resultado obtido e o número de passos executados pelo reconhecedor.

Linguagem	Entrada	Esperado	Obtido	Passos
LR	123.456.789-00	aceita	aceita	14
LR	000.000.000-00	aceita	aceita	14
LR	987.654.321-99	aceita	aceita	14
LR	12345678900	rejeita	rejeita	11
LR	12.345.678-90	rejeita	rejeita	13
LR	123.456.789.00	rejeita	rejeita	14
LLC	(a+b)	aceita	aceita	6
LLC	((x+y)*z)	aceita	aceita	10
LLC	{[(x+y)]}	aceita	aceita	10
LLC	((a+b)	rejeita	rejeita	7
LLC	[a+b))	rejeita	rejeita	5
LLC	{(a+b)}	rejeita	rejeita	6
R	#	aceita	aceita	2
R	0#0	aceita	aceita	10
R	101#101	aceita	aceita	44
R	0#1	rejeita	rejeita	3
R	101#100	rejeita	rejeita	29
R	01#011	rejeita	rejeita	24

Todos os testes retornaram o resultado esperado. O DFA apresentou o menor número de passos porque faz apenas uma leitura linear da entrada. O PDA precisou de controle adicional da pilha para verificar os delimitadores. A Máquina de Turing consumiu mais passos, pois precisa deslocar a cabeça pela fita várias vezes para comparar as duas partes da string.

6. Execução passo a passo

6.1 DFA: entrada aceita

Entrada: 123.456.789-00

Passo	Símbolo lido	Estado anterior	Estado seguinte
1	1	q0	q1
2	2	q1	q2
3	3	q2	q3
4	.	q3	q4
5	4	q4	q5
6	5	q5	q6
7	6	q6	q7
8	.	q7	q8
9	7	q8	q9
10	8	q9	q10

11	9	q10	q11
12	-	q11	q12
13	0	q12	q13
14	0	q13	q14

Resultado: aceita, pois a leitura terminou no estado final q14.

6.2 DFA: entrada rejeitada

Entrada: 12345678900

Passo	Símbolo lido	Estado anterior	Estado seguinte
1	1	q0	q1
2	2	q1	q2
3	3	q2	q3
4	4	q3	q_err

Resultado: rejeita, pois no estado q3 era esperado um ponto, mas foi lido um dígito.

6.3 PDA: entrada aceita

Entrada: (a+b)

Passo	Símbolo lido	Ação	Pilha
1	(	empilha (	Z(
2	a	mantém pilha	Z(
3	+	mantém pilha	Z(
4	b	mantém pilha	Z(
5	)	desempilha (	Z
6	ϵ	aceita	Z

Resultado: aceita, pois todos os delimitadores foram fechados corretamente e a pilha terminou apenas com o símbolo de fundo Z.

6.4 PDA: entrada rejeitada

Entrada: {(a+b)}

Passo	Símbolo lido	Ação	Pilha
1	{	empilha {	Z{
2	(	empilha (	Z{(
3	a	mantém pilha	Z{(
4	+	mantém pilha	Z{(
5	b	mantém pilha	Z{(
6	]	fechamento incompatível	Z{(

Resultado: rejeita, pois o símbolo] só poderia desempilhar [, mas o topo da pilha era (.

6.5 Máquina de Turing: entrada aceita

Entrada: 101#101

Etapa	Descrição	Fita resumida
1	Marca o primeiro 1 da esquerda	X01#101
2	Procura o primeiro símbolo não marcado após # e encontra 1	X01#101
3	Marca o correspondente no lado direito	X01#X01
4	Retorna ao início da fita	X01#X01
5	Marca o 0 da esquerda	XX1#X01
6	Procura o próximo símbolo não marcado no lado direito e encontra 0	XX1#X01
7	Marca o correspondente	XX1#XX1
8	Retorna ao início da fita	XX1#XX1
9	Marca o último 1 da esquerda	XXX#XX1

10	Procura o próximo símbolo não marcado no lado direito e encontra 1	XXX#XX1
11	Marca o correspondente	XXX#XXX
12	Verifica que não sobraram símbolos não marcados	XXX#XXX

Resultado: aceita, pois todos os símbolos da primeira parte possuem correspondentes iguais na segunda parte.

6.6 Máquina de Turing: entrada rejeitada

Entrada: 101#100

Etapa	Descrição	Fita resumida
1	Marca o primeiro 1 da esquerda	X01#100
2	Encontra 1 correspondente no lado direito	X01#X00
3	Marca o 0 da esquerda	XX1#X00
4	Encontra 0 correspondente no lado direito	XX1#XX0
5	Marca o último 1 da esquerda	XXX#XX0
6	Procura 1 correspondente, mas encontra 0	XXX#XX0

Resultado: rejeita, pois o símbolo esperado era 1, mas o símbolo encontrado no lado direito foi 0.

7. Comparação entre os três níveis

Os três reconhecedores mostram bem a diferença entre os níveis $LR \subsetneq LLC \subsetneq R$. O DFA resolve um padrão fixo, o PDA já precisa de memória para controlar aninhamentos, e a Máquina de Turing consegue comparar duas partes da entrada usando marcações na fita.

Característica	DFA	PDA	Máquina de Turing
Classe	Linguagem Regular	Linguagem Livre de Contexto	Linguagem Recursiva
Memória auxiliar	Não possui	Pilha	Fita
Exemplo implementado	CPF	Delimitadores balanceados	w#w
Poder computacional	Menor	Intermediário	Maior
Tipo de processamento	Leitura linear	Controle de aninhamento	Leitura, escrita e deslocamento
Número de passos observado	Menor	Intermediário	Maior

Na prática, o aumento do poder computacional aparece junto com a necessidade de mais memória. O DFA depende apenas do estado atual, o PDA utiliza uma pilha para controlar estruturas aninhadas, e a Máquina de Turing usa uma fita que permite marcar, comparar e visitar partes da entrada.

8. Conclusão e limitações

Com os três reconhecedores, foi possível observar na prática a diferença entre linguagens regulares, livres de contexto e recursivas. O DFA resolveu um problema de formato fixo, o PDA lidou com balanceamento usando pilha, e a Máquina de Turing comparou duas partes de uma string usando marcações e deslocamentos na fita.

Como limitações, o CPF foi validado apenas quanto ao formato textual, sem cálculo dos dígitos verificadores. O PDA não realiza análise sintática completa de expressões, apenas verifica se os delimitadores estão balanceados. Já a Máquina de Turing foi construída especificamente para a linguagem $L = \{w\#w \mid w \in \{0,1\}^*\}$, não sendo uma implementação genérica para qualquer linguagem recursiva.